

CITS5508 Practice Exam — Chapter 6 — ANSWER KEY

Question 1.

- (a) (3 marks) Proportions: $A = 0/50 = 0$, $B = 40/50 = 0.8$, $C = 10/50 = 0.2$. $Gini = 1 - (0^2 + 0.8^2 + 0.2^2) = 1 - (0 + 0.64 + 0.04) = 1 - 0.68 = \mathbf{0.32}$.
- (b) (4 marks) Entropy = $-0.8 \cdot \log_2(0.8) - 0.2 \cdot \log_2(0.2)$ (the 0-probability class contributes nothing). $\log_2(0.8) \approx -0.3219$, $\log_2(0.2) \approx -2.3219$. $= -0.8(-0.3219) - 0.2(-2.3219) = 0.2575 + 0.4644 \approx \mathbf{0.722}$.
- (c) (2 marks) If all 50 belong to one class, $Gini = 1 - 1^2 = \mathbf{0}$. Such a node is **pure**.
- (d) (3 marks) They usually give **similar trees**. **Gini** is **slightly faster** (no logarithm) and is the scikit-learn **default**. When they differ, Gini tends to **isolate the most frequent class** in its own branch, whereas entropy tends to produce **slightly more balanced** trees.
-

Question 2.

- (a) (4 marks) - Left (30 A, 10 B of 40): $Gini = 1 - (30/40)^2 - (10/40)^2 = 1 - 0.5625 - 0.0625 = \mathbf{0.375}$. - Right (10 A, 50 B of 60): $Gini = 1 - (10/60)^2 - (50/60)^2 = 1 - 0.0278 - 0.6944 \approx \mathbf{0.278}$.
- (b) (3 marks) Cost = $(40/100) \cdot 0.375 + (60/100) \cdot 0.278 = 0.15 + 0.1667 \approx \mathbf{0.317}$.
- (c) (3 marks) CART is **greedy** because it picks the **best split at the current node** and never reconsiders it in light of deeper splits (no look-ahead). Finding the **optimal** tree is **NP-complete** ($\approx O(\exp(m))$) $\rightarrow$ intractable even for small datasets, so we settle for a “reasonably good” greedy solution.
- (d) (2 marks) **Prediction:** $O(\log_2 m)$ (traverse root $\rightarrow$ leaf, independent of #features). **Training:** $O(n \cdot m \cdot \log_2 m)$.
-

Question 3.

- (a) (4 marks) **Nonparametric** = the number of parameters is **not fixed before training**; the tree’s structure is free to adapt to the data. Because of this freedom it can **stick arbitrarily close to the training data**, capturing noise $\rightarrow$ **overfitting** unless constrained. (Contrast: a parametric model like linear regression has a fixed parameter count.)
- (b) (4 marks) Any four, with direction (to reduce overfitting): - max_depth — **decrease** - max_leaf_nodes — **decrease** - max_features — **decrease** - min_samples_split — **increase** - min_samples_leaf — **increase** - min_weight_fraction_leaf — **increase** (Rule: increase min_*, decrease max_*.)
- (c) (4 marks) **Pruning:** grow the full (unconstrained) tree, then **delete nodes** whose purity gain is not **statistically significant**. A test such as the χ^2 test estimates the probability (**p-value**) that the observed improvement is due to **chance** (the null hypothesis); if the p-value exceeds a threshold (e.g. 5%), the node’s children are **pruned**. Continue until no more unnecessary nodes remain.
-

Question 4.

(a) (4 marks) **White-box** models (decision trees) are **interpretable** — you can read off the simple if/then rules and explain exactly why a prediction was made (rules can even be applied by hand). **Black-box** models (random forests, neural nets) give great predictions but it's hard to explain *why* in human terms. This matters for **trust, debugging, and fairness** (e.g. ensuring decisions aren't unfair).

(b) (4 marks) A tree **traverses to the leaf** the instance falls into and returns the **ratio of each class among that leaf's training instances** (e.g. leaf with [0, 49, 5] → [0%, 90.7%, 9.3%]). Every instance landing in the **same leaf** gets the **same** estimate because the tree cannot distinguish positions **within** a leaf's region — the prediction is constant across that region.

(c) (4 marks) **Feature scaling** (standardisation/normalisation) is **not** needed for decision trees but **is** needed for SVMs and k-NN. Trees split one feature at a time using thresholds, so monotonic rescaling doesn't change the splits; SVMs and k-NN rely on **distances/geometry**, so a large-range feature would dominate and distort the model unless features are scaled.

Question 5.

(a) (4 marks) Decision trees make **orthogonal splits** (perpendicular to an axis), so they are **sensitive to the data's orientation**: a simple linearly-separable dataset splits cleanly when axis-aligned but needs a **convoluted staircase** boundary after a 45° rotation, generalising worse. **Mitigation**: scale the data and apply **PCA** to rotate it toward axis-aligned structure (reducing feature correlation).

(b) (4 marks) **High variance** = small changes to the data or hyperparameters (and even the stochastic feature selection on the same data) produce **very different trees**. An **ensemble** averages the predictions of **many** trees (a **random forest**), which **reduces the variance** substantially while keeping similar bias → better generalisation.

(c) (4 marks)

```
from sklearn.tree import DecisionTreeClassifier

tree_clf = DecisionTreeClassifier(max_depth=3, min_samples_leaf=10, random_state=42)
tree_clf.fit(X_train, y_train)
print(tree_clf.feature_importances_)
```

(3 marks for the regularised classifier + fit; 1 mark for printing feature_importances_.)